TASK-DRIVEN LIDAR POINT CLOUD COMPRESSION AND TRANSMISSION FOR AUTONOMOUS DRIVING

A Thesis

Presented to

The Faculty of the Department of Computer Engineering

San José State University

In Partial Fulfillment

of the Requirements for the Degree

Master of Science

by

Su Hyun Kim

May 2026

© 2026

Su Hyun Kim

ALL RIGHTS RESERVED

The Designated Thesis Committee Approves the Thesis Titled

TASK-DRIVEN LIDAR POINT CLOUD COMPRESSION AND TRANSMISSION FOR AUTONOMOUS DRIVING

by

Su Hyun Kim

APPROVED FOR THE DEPARTMENT OF COMPUTER ENGINEERING

SAN JOSÉ STATE UNIVERSITY

May 2026

Jun Liu, Ph.D. Department of Computer Engineering

ABSTRACT

TASK-DRIVEN LIDAR POINT CLOUD COMPRESSION AND TRANSMISSION FOR AUTONOMOUS DRIVING

by Su Hyun Kim

LiDAR sensors provide precise geometric context for autonomous vehicles, but the sheer volume of points emitted by modern spinning and solid-state units overwhelms embedded storage and wireless backhauls. This thesis tackles that bottleneck by treating LiDAR compression as a staged representation-learning problem. Rather than relying on auxiliary cameras or handcrafted codecs, the proposed pipeline stays within the LiDAR modality and progressively increases modeling difficulty so that each stage delivers a verifiable research milestone.

Stage 1 projects each sweep onto a cylindrical range image and trains a residual convolutional autoencoder with quantization-aware latents to establish a transparent baseline. Stage 2 adds temporal residual coding by warping the previously reconstructed frame toward the incoming sweep so that only the innovation needs to be transmitted. Stage 3 hardens the representation against sparse or missing data via masking and generative completion, encouraging the decoder to reconstruct dense geometry from aggressive subsampling. An optional Stage 4 allocates bitrate according to downstream task importance, ensuring that regions containing safety-critical agents are preserved even when bandwidth is scarce.

Collectively, these stages yield a LiDAR-only codec that spans single-frame compression, inter-frame prediction, and learned completion within one cohesive framework. The design emphasizes reproducibility (each stage can be evaluated independently), practical deployment constraints (closed-form bandwidth accounting through entropy coding), and perception alignment (losses applied in both range-image and 3D spaces). The resulting roadmap provides a concrete path for improving

reconstruction quality while steadily lowering the communication burden of LiDAR sensing.

ACKNOWLEDGMENTS

I would like to thank my advisor, Jun Liu, for being the greatest advisor in the world.

I would also like to thank Cheryl R. Cowan, SJSU Graduate Studies Associate, for assisting me in creating this LaTeX template and ensuring that it accurately follows the Graduate Studies thesis style requirements.

Last, but not least, I would like to thank my family. Without their support, I would have never been able to complete this thesis or pursue my dreams.

TABLE OF CONTENTS

List of Tables ix

List of Figures x

1 Introduction 1

2 Literature Review 3
2.1 Surveys and Evaluation Frameworks 3
2.2 Learned Point Cloud Compression 3
2.3 Task-Driven and Scene-Aware Pipelines 4
2.4 Real-Time Robotics and Mapping Considerations 5
2.5 Operational Integration in Mobility Services 5

3 Proposed Method 7
3.1 Stage 1: Range-Image Autoencoder 7
3.2 Stage 2: Spatio-Temporal Residual Coding 7
3.3 Stage 3: Masked Representation Learning and Completion 8
3.4 Stage 4: Task-Aware Bit Allocation 8

4 Sections and Headings 10
4.1 Second Section Level 10
4.1.1 Third Section Level 11
4.1.1.1 Fourth section level 11

5 Example Sections 12

6 Mini-introduction to LaTeX 13
6.1 Equations 13
6.2 Figures 13
6.3 Tables 14
6.4 Algorithms 14
6.5 Location of Floats 16
6.6 Citations 17

7 Writing Advice 18

8 Future Work 19

9 Conclusions 20

Literature Cited 21

Appendix A: What Should Be Included in Appendices 24
A.1 Subsections in Appendices 24

Appendix B: Title of Another Appendix 26

LIST OF TABLES

Table 1. Dataset Characteristics 14

Table 2. Graph and Projection Notation 16

LIST OF FIGURES

Fig. 1. Simplified LiDAR-only compression pipeline showing the range-image encoder (Stage 1), temporal residual context (Stage 2), masked completion decoder (Stage 3), and optional task-aware bitrate control (Stage 4). 9

Fig. 2. Project architecture. 14

Fig. 3. Distribution of sample non-zeros. 25

Fig. 4. Speedup comparison of our method against baselines..... 26

1 INTRODUCTION

Autonomous driving stacks rely on LiDAR to capture metrically accurate 3D geometry regardless of illumination or weather. Successive sensor generations increase channel counts and angular resolution, producing tens of millions of points per second that quickly saturate on-board compute, storage, and wireless uplinks. Sustaining fleet-scale perception therefore demands LiDAR representations that aggressively compress data yet preserve the structure required for downstream reasoning.

Range-image projection offers the most accessible entry point because it converts irregular point clouds into dense $H \times W \times C$ tensors that conventional convolutional autoencoders can process. Learned codecs such as RICNet incorporate entropy models and spatial priors tailored to projected sweeps, while octree and predictive schemes extend rate–distortion trade-offs within a single frame [1]. Temporal models like U-Motion demonstrate that inter-frame residual coding can further reduce bitrate [2], but they seldom connect compression with the representation-learning objectives needed to reconstruct masked or missing regions. Recent surveys underscore the absence of a LiDAR-only roadmap that spans from intuitive baselines to generative completion while remaining faithful to real-time constraints [3].

This thesis answers that need through a staged LiDAR-only research agenda that increases modeling difficulty in controlled increments. **Stage 1** converts each sweep into a 2D range image and trains a residual CNN autoencoder with quantization-aware latent bottlenecks, following RICNet-style practices that provide a reproducible baseline [1]. **Stage 2** injects spatio-temporal reasoning by conditioning the encoder on a motion-compensated reconstruction of the previous frame and coding only the residual novelty inspired by U-Motion [2]. **Stage 3** recasts compression as representation learning: aggressive masking and down-sampling force the network to infer dense geometry from sparse evidence by relying on discrete codebooks in the spirit of UltraLiDAR and LiDAR completion [4]. An optional

Stage 4 steers entropy coding toward task-critical regions (e.g., vehicles or pedestrians) so that machine-vision accuracy is preserved even at low bitrates [5].

By foregrounding representation learning and reconstruction quality at every layer, the roadmap provides (i) a transparent LiDAR-only baseline, (ii) a temporal compression module that bridges motion dynamics with rate–distortion objectives, (iii) a generative completion head aligned with reconstruction fidelity, and (iv) an evaluation strategy that links bandwidth savings to downstream perception. Each successive chapter builds on the previous one, ensuring that the final system meets real-time deployment constraints while advancing the thesis goals of representation learning and reconstruction-centric LiDAR compression.

The thesis therefore contributes (1) a cleanly reproducible range-image codec that serves as a reference point for future LiDAR compression work, (2) a spatio-temporal residual module that converts the baseline into a predictive coder capable of exploiting motion cues, (3) a masking-driven completion strategy that brings representation learning objectives directly into the codec, and (4) a task-aware bitrate controller that preserves perception-critical regions under tight bandwidth budgets. Together these elements translate the staged roadmap into a concrete system design that can be implemented, analyzed, and iterated upon throughout the remainder of this document.

2 LITERATURE REVIEW

Autonomous mobility platforms increasingly depend on dense 3D sensing, yet the raw bandwidth and storage demands of LiDAR streams quickly overwhelm on-board compute and communication channels. This section situates the proposed research within four complementary threads: (i) surveys that map the evolving design space of LiDAR compression and its evaluation protocols, (ii) learned codecs that push rate–distortion and task-awareness, (iii) pipelines that align compression with scene semantics and localization objectives, and (iv) systems research that couples perception bandwidth with mobility operations. These threads collectively frame the staged LiDAR-only roadmap introduced in section 1.

2.1 Surveys and Evaluation Frameworks

Recent syntheses provide a structured view of how LiDAR compression has matured from handcrafted quantization rules into hybrid learned systems. Abbasi et al. chart the progression from classical octree pruning to neural entropy models while emphasizing that compression choices impact perception tasks in subtle ways [6]. Wang et al. extend that perspective by underscoring how emerging datasets, evaluation metrics, and multimodal sensor suites reshape codec requirements [7]. Complementing these qualitative taxonomies, Martins et al. deliver quantitative evidence that both MPEG baselines and recent neural codecs can severely degrade object detection unless detector training pipelines are adapted, particularly once bitrates dip below six bits per point [8]. Together, these surveys highlight two persistent challenges: jointly optimizing compression with downstream analytics, and developing benchmarks that measure more than geometric fidelity.

2.2 Learned Point Cloud Compression

Classical lossless codecs, most notably LASzip, still underpin many production LiDAR workflows because they preserve the exact LAS record layout while shrinking storage footprints by an order of magnitude [9]. Against that backdrop, learning-based codecs have

become the focal point for reducing bitrate without sacrificing machine perception. Range image entropy models such as RICNet [1] and adaptive octree depth predictors [10] tailor spatial priors to sparsely sampled LiDAR scans, achieving lower entropy than handcrafted predictors. Temporal models deepen these gains: U-Motion [2] fuses coarse-to-fine motion propagation within a U-shaped architecture to stabilize point cloud video coding, while LPCM [11] frames LiDAR compression as predictive coding in latent space, sharing parameters across multiple quantization stages. Cross-modal conditioning further enriches the latent representations—ViewPCGC [12] injects perspective imagery to guide contour preservation, and ROI-guided compression allocates additional bits to semantically critical regions so that both human operators and downstream networks maintain accuracy [5]. Inter-frame feature warping in TIP's geometry coder [13] and multi-modal backbones such as MSeg3D [14] confirm that motion cues and sensor fusion must be embedded directly into the codec to fully exploit redundancies.

2.3 Task-Driven and Scene-Aware Pipelines

Although generic learned codecs yield impressive rate–distortion curves, several works demonstrate that aligning compression with task objectives can deliver larger practical benefits. Sun et al. incorporate semantic segmentation to steer bitrate toward dynamic, safety-critical agents, showing improved downstream perception under the same bitrate budget [15]. Martins et al. further reveal that retraining detectors on reconstructed point clouds recovers accuracy lost to quantization, effectively advocating for codebooks that are co-designed with the perception stack [8]. Beyond perception, map-focused studies evaluate how alternative scene representations affect localization. Chang et al. show that carefully chosen compressive map formats can strike favorable trade-offs between storage and registration robustness, enabling lighter-weight maps without sacrificing re-localization accuracy [16]. Collectively, these studies motivate codecs that expose explicit controls for allocating bits to semantic classes, structural priors, or map features depending on the intended downstream application.

2.4 Real-Time Robotics and Mapping Considerations

From a systems viewpoint, the feasibility of any codec hinges on latency, compute footprint, and integration with online SLAM. The O-POCO pipeline exemplifies early efforts to compress feature maps on-the-fly, prioritizing relocalization accuracy while thinning redundant 3D points in visual odometry loops [17]. Tu et al. investigate recurrent neural networks that directly operate on raw LiDAR packets, reporting negligible SLAM degradation even as storage costs drop substantially [18]. Recent frameworks push further toward deployment by integrating adaptive bitrate control and cloud-assisted processing. Feng et al. demonstrate real-time spatio-temporal compression that maintains over-10 Hz throughput, whereas Cao et al. add QoE-based bitrate modulation to remain resilient under fluctuating wireless bandwidth, ensuring perception tasks continue to receive timely updates [19], [20]. These systems collectively argue that codec design must balance predictive accuracy with computational and networking constraints inherent to field robots.

2.5 Operational Integration in Mobility Services

Efficient LiDAR compression also influences upstream decision-making infrastructure in mobility services where sensing, forecasting, and control pipelines are tightly coupled. Large-scale order dispatch algorithms, such as the KDD'18 framework deployed at Didi Chuxing, rely on timely state estimates to plan resource allocation across cities [21]. More recent learning-augmented optimization incorporates probabilistic acceptance models and reinforcement learning to personalize slack times in high-capacity ride pooling, again assuming reliable sensory inputs [22]. Compression frameworks that can stream rich 3D context without saturating vehicular networks therefore become enablers for these analytics pipelines. High-fidelity yet bandwidth-aware LiDAR data feeds improve situational awareness for fleet control, which closes the loop between perception research and operational logistics [20].

Taken together, the literature underscores a clear direction: future LiDAR compression must be adaptable to application semantics, optimized jointly with machine perception models, and cognizant of the communication and decision-making infrastructure it supports. The techniques surveyed here motivate the architectural choices and evaluation criteria adopted in the remainder of this thesis.

3 PROPOSED METHOD

Fig. 1 summarizes the LiDAR-only compression pipeline developed in this work. Each input sweep is first projected into a cylindrical range image, optionally masked to simulate sparse sampling, and then passed through a temporal fusion module that aligns previously reconstructed frames before encoding. Latent features are quantized, entropy coded into a compact bitstream, and later decoded to reconstruct the dense range image and corresponding point cloud. Losses defined on both the 2D range grid and reconstructed 3D geometry supervise the entire pipeline so that compression factors can be tuned without sacrificing reconstruction fidelity or downstream perception accuracy.

3.1 Stage 1: Range-Image Autoencoder

Stage 1 focuses on building a reproducible baseline inspired by RICNet [1]. Input point clouds are converted into $H \times W \times C$ range images storing range, reflectivity, and auxiliary channels. A lightweight 2D convolutional encoder with residual blocks downsamples the tensor into a compact latent grid. Uniform or vector-quantized bottlenecks convert latent activations into discrete symbols that are entropy coded into a bitstream for transmission or storage. Decoding mirrors this path via transpose convolutions, producing a reconstructed range image that is back-projected into a point cloud and supervised with range-domain ℓ_1 loss plus Chamfer Distance.

3.2 Stage 2: Spatio-Temporal Residual Coding

Stage 2 extends the baseline by ingesting the previously decoded frame $\hat{R}_{t-1}$, estimating a 2D motion field between $\hat{R}_{t-1}$ and the current projection, and warping features so that only residual innovation is compressed, echoing U-Motion's coarse-to-fine strategy [2]. The temporal fusion module shares weights with the encoder so that recurrent state grows implicitly within the latent grid. Training alternates between key-frame encodings that refresh the context and residual frames that rely heavily on the warped features, yielding bitrate reductions without degrading reconstruction quality.

3.3 Stage 3: Masked Representation Learning and Completion

Stage 3 reframes compression as representation learning by aggressively masking beams, pixels, or entire regions prior to encoding. The decoder is upgraded into a generative head (e.g., a VQ-VAE or diffusion-inspired module) that learns discrete codebooks tailored to LiDAR structure, enabling it to hallucinate dense geometry conditioned on sparse evidence, similar to UltraLiDAR-style completion [4]. Perceptual losses computed on range images, Chamfer Distance in 3D, and temporal smoothness regularizers on the latent code encourage faithful reconstruction even when more than half of the input is missing.

3.4 Stage 4: Task-Aware Bit Allocation

The final, optional stage injects perception awareness into the entropy model by reweighting latent channels associated with regions of interest (ROI) such as vehicles or pedestrians, akin to ROI-guided codecs [5]. Detection or segmentation losses back-propagate through the decoder so that the codec learns to preserve details that matter for downstream autonomy tasks while allowing benign regions to receive fewer bits. The same feedback channels can inform adaptive bitrate control during deployment, ensuring that limited bandwidth is devoted to safety-critical content.

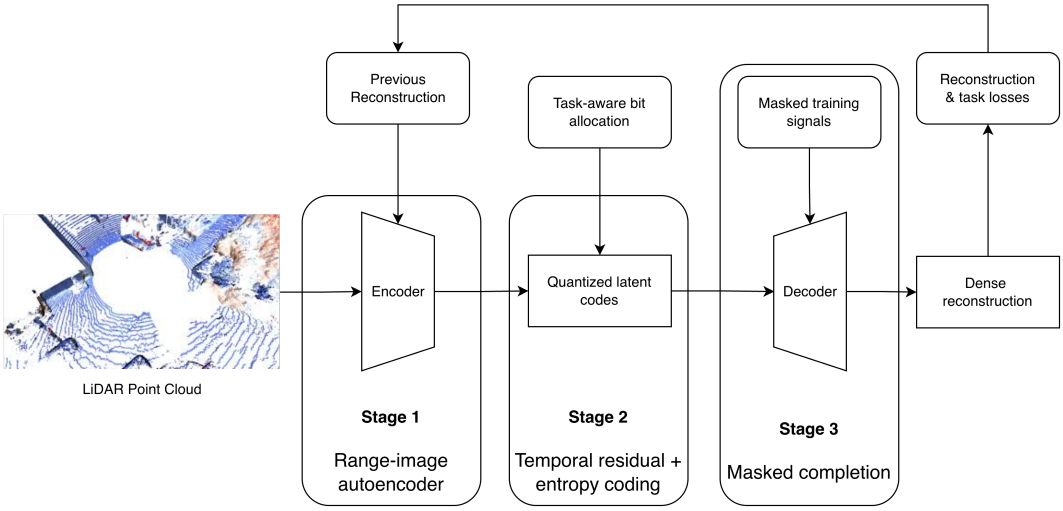

Fig. 1: Simplified LiDAR-only compression pipeline showing the range-image encoder (Stage 1), temporal residual context (Stage 2), masked completion decoder (Stage 3), and optional task-aware bitrate control (Stage 4).

4 SECTIONS AND HEADINGS

A thesis is generally broken down into sections, each section encompassing a different area of the work. For example, the first section in a thesis is generally an “Introduction”, which lays out what the research is about. The top-level sections in the thesis are called chapters and always start at the top of a page. When referring to a top-level section, one should mention it as Chapter 1, where the number is the section/chapter number. The numbers, as with all references in the thesis, should never be written out. Rather, use LaTeX reference macros, which will be automatically replaced with appropriate numbers when compiling the thesis document. Chapter 6, as well as the source code of this document, has many examples of references.

A thesis section can be further broken down into subsections. One should use at most 4 section levels in a thesis. This paragraph is in the 1st section level, “section”. We will create 3 more subsection levels. The 2nd section level is created using the “subsection” macro. The 3rd section level is created using the “subsubsection” macro. However, the 4th section level is created using the “paragraph” macro. Note that the IEEE paragraph style is slightly different than the section styles, as the paragraph caption will be in-line with the text of the section and will be automatically ended with a colon. Students should note that more than 3 subheadings (4 section levels) becomes confusing to the reader and should thus avoid further breakdown of paragraph sections.

Note that the section header is written in title case, though it appears in the text as uppercase. Students should not deviate from using title case, as it will change the appearance of the section title in the table of contents. The section title should be written in title case.

4.1 Second Section Level

This is an example of the 2nd level of a section. The subsection title should be written in title case.

4.1.1 Third Section Level

This is an example of the 3rd level of a section. The subsubsection title should be written in title case.

4.1.1.1 Fourth section level:

This is an example of the 4th level of a section. Note that the paragraph title should be written in sentence case, unlike the higher level titles.

5 EXAMPLE SECTIONS

In this chapter we discuss a possible set of chapters for a thesis. The example is somewhat specific to the author's field of research (Data Science). Students should, in general, consult with their adviser on the list of chapters that they should include in their thesis.

6 MINI-INTRODUCTION TO LATEX

This chapter needs to be filled out. For now, it contains some very basic examples.

6.1 Equations

Equations and mathematical formulas can be included in-line, within the text, for example when explaining that $\|\mathbf{x}\|_2$ signifies the ℓ_2 norm of a vector, as long as the included math formula or symbols are short (less than half a line). For longer formulas, or formulas that need to be referenced later in the text, one should use the `equation` macro or related macros, e.g., `align`. For example, the dot-product of two vectors, **x** and **y**, is defined as

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^T \mathbf{y} = \sum_{j=1}^{m} x_j \times y_j. \tag{1}$$

Note that equations can be assigned labels and referenced throughout the text. However, unlike figures and tables (see next sections), the equation should be defined prior to its reference. Now that we can defined the formula for the vector dot-product, we can now refer to Equation 1 to remind readers of its definition.

6.2 Figures

Figures should contain a caption at the bottom. The caption is a sentence or phrase that describes the content of the figure. As such, it should start with a capital letter and end with a period. The caption is not a title, so subsequent words after the first word should not be capitalized unless they are proper names. A figure should be introduced in the text prior to its inclusion in the text. When referring to the figure, make use of the `\figurename` macro, provided by the IEEE style, to ensure style compliance. Additionally, use a non-breaking space character to connect the Fig. text with the number assigned to the figure reference. This will ensure that Fig. and the reference will be included on the same line. For example, Fig. 2 introduces our project architecture. Note that font sizes in figures and tables may be smaller than the rest of the thesis. However, they should be readable.

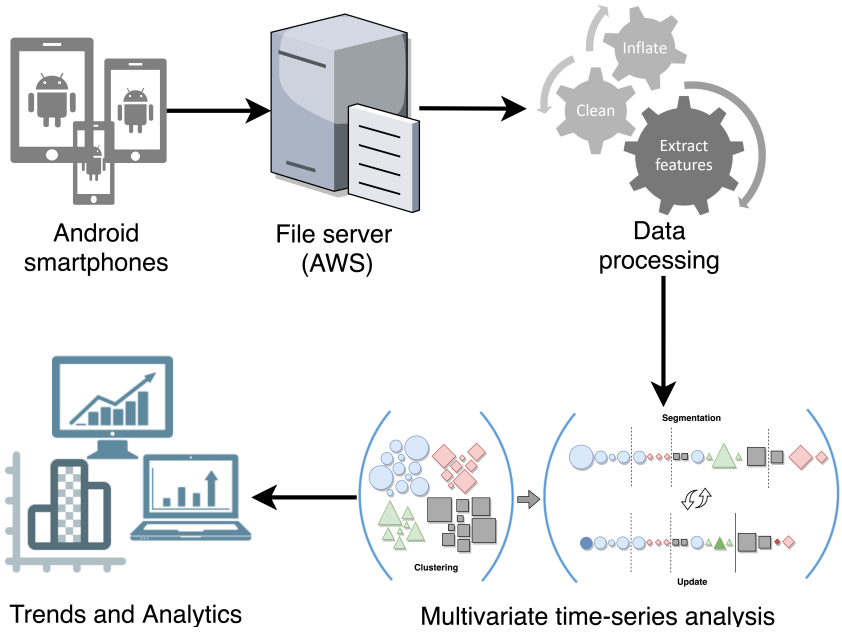

Fig. 2: Project architecture.

6.3 Tables

Tables should contain a caption at the top. Unlike figures, the caption in the table is the table title, and it should be capitalized accordingly. The table should be presented in the text prior to its inclusion in the text. Moreover, references to tables should follow the same directions as those for figures, except use the `\tablename` macro. For example, Table 1 introduces statistics about the dataset used in our analysis.

Table 1: Dataset Characteristics

Name	Duration (h)	Activities	Subjects	Sensors	Frequency (Hz)
JSI-ADL	18.8	14	10	2	50
REALDISP	9	33	17	2	50
UCI-HAR	1.5	6	30	2	50
Ours	1500	14	24	2	20

6.4 Algorithms

LaTeX provides powerful macros for writing algorithms. Note that labels can be included within the algorithm, which allows the writer to refer to specific lines in the algorithm. The

algorithm should be introduced first. Note that the label referring to the whole algorithm is placed right after the caption. Similar to a table, the caption is a title and should be capitalized accordingly. For example, Algorithm 1 describes the process for producing a normalized 2-hop left-projection of a bipartite graph described by the vertex sets V^1 and V^2 and the edge set E. Long algorithms (greater than half a page) should be relegated to the Appendix.

Algorithm 1 The Normalized 2-hop Left-Projection Algorithm

1: **procedure** PROJECT(V^1, V^2, E) ▷ Construct the left projection G^1
    // Initialize and compute degree vectors.
2: **for** $(v_i^1, v_j^2) \in E$ **do** ▷ Count neighbors
3: $\lambda_n(i) \leftarrow \lambda_n(i) + 1$
4: $\lambda_m(j) \leftarrow \lambda_m(j) + 1$
    // SpGEMM
5: **for** $v_i^1 \in V^1$ **do**
6: $acc(:) \leftarrow 0$ ▷ Initialize output row accumulator data structure
7: **for** $v_k^2 \in \Gamma(v_i^1)$ **do**
8: $l \leftarrow (1/\lambda_n(i)) \times \times w_{i,k} \times (1/\lambda_m(k))$
9: **for** $v_j^1 \in \Gamma(v_k^2)$ **do**
10: **if** $v_j^1 \neq v_i^1$ **then** ▷ Avoid computing output diagonal
11: $acc(j) \leftarrow acc(j) + l \times w_{j,k}$
12: $C(i,:) = acc(:)$
    // Normalize rows of C
13: **for** $i = 1, \ldots, |V^1|$ **do**
14: $n_i \leftarrow 0$
15: **for** $j = 1, \ldots, |V^2|$ s.t. $C(i,j) > 0$ **do**
16: $n_i \leftarrow n_i + \text{abs}(C(i,j))$
17: **for** $j = 1, \ldots, |V^2|$ s.t. $C(i,j) > 0$ **do**
18: $C(i,j) \leftarrow C(i,j)/n_i$
19: **return** C ▷ Edge set weight matrix for G^1

After introducing the algorithm, one can then refer to sections of the algorithm in the process of describing it in the text. For example, note that line 11 denotes the accumulation of projection weights for a vertex v_i^1 in V^1. For theses that contain many mathematical symbols, it is a good idea to provide the reader with a reference for the meaning of these symbols. This table is generally included in the preliminary chapters, after the introduction. We will include an example here. Table 2 provides a reference for notation used throughout the paper that the algorithm above is included in.

Table 2: Graph and Projection Notation

Symbol	Meaning
G	A graph topology.
V^1, V^2	The left and right node partitions in a bipartite graph.
E	The set of edges in G.
$\|X\|$	The cardinality of set X.
$v_i^1 \in V^1$	A vertex in the left node partition V^1.
$v_j^2 \in V^2$	A vertex in the right node partition V^2.
$(v_i^1, v_j^2, w_{i,j}) \in E$	The edge connecting nodes v_i^1 and v_j^2 with weight $w_{i,j}$ in graph G.
$\Gamma(v_i^1)$	The set of vertices in V^2 adjacent to v_i^1, i.e., its neighborhood.
λ_n, λ_m	Node degree vectors for the left and right partitions.
$A \in \mathbb{R}^{n \times m}$	The edge weight matrix for the bipartite graph G; $A(i,j) = w_{i,j}$.
B	Weighted adjacency matrix for graph G.
$G^1 = (V^1, E^1)$	The left projection graph.
$G^2 = (V^2, E^2)$	The right projection graph.
$G^b = (V, E^b)$	The biprojection graph.
$(v_i^1, v_j^1, s_{i,j}^1) \in E^1$	The edge connecting v_i^1 and v_j^1 with weight $s_{i,j}^1$ in the left projection graph.
W^1	Edge weight matrix for the left projection graph; $W^1(i,j) = s_{i,j}^1$.
W^2	Edge weight matrix for the right projection graph; $W^2(i,j) = s_{i,j}^2$.
W	Weighted adjacency matrix for the bi-projection graph of G.

6.5 Location of Floats

Floats (e.g., algorithms, tables and figures) should not follow the IEEE standard of being placed at the top or bottom of the page. As per the GUP thesis guidelines, they must appear right after their reference and not break any paragraphs, i.e., they should be placed in the text, after the paragraph in question. Use the location hint `[!htb]` to let LaTeX know the float should be placed in between paragraphs. If the float takes up an entire page and the last paragraph on the current page does not fit in its entirety, it should be moved after the float page to avoid breaking the continuity of the paragraph, leaving some blank lines at the end of the current page.

Pages that contain only floats will have floats vertically centered, which will produce additional blank lines between the floats in the page or between a float and the top and bottom of the page. The thesis guidelines specifically prohibit additional white space in the thesis. This case and the one discussed in the previous paragraph are the only acceptable reasons for having extra blank lines in the text.

6.6 Citations

When citing papers, it is advisable to collect BibTeX records for the papers that you wish to cite, which will be automatically processed into the correct citation style by the LaTeX/BibTeX compiler. Citing a work by Anastasiu and Karypis [23], for example, is as simple as writing "Anastasiu and Karipis~\cite{anastasiu-dsaa2016}," where *anastasiu-dsaa2016* is the key associated with the paper's BibTeX record in the *references.bib* file. Note the non-breaking space character (~) between the word before the citation and the *\cite* command. It gets translated into a space, and does not allow LaTeX to print the citation on a different line than the connecting word. This, of course, does not work if you put a space between the word and the non-breaking space character, e.g., "Karypis ~\cite{anastasiu-dsaa2016}".

When citing papers with a single author, one should refer to the work by citing the author's last name. For example, Anastasiu [24] used a combination of heuristic candidate selection and search space filtering to efficiently identify approximate nearest neighbors. When the work has two authors, both authors should be referenced by their last name, as in the previous example of the work by Anastasiu and Karypis [23]. Finally, if there are more than two authors, one should reference the first author's last name, followed by the phrase *et al.*, which is short for the Latin *et alia* and means "and others". Note the placement of the period. As an example, Kapoor et al. [25] discuss the benefits of competitive active learning in engineering education. Note that it is not necessary to mention authors by last name when listing multiple works as examples or evidence of a statement. For example, many recent works have developed filtering-based methods for nearest neighbor graph construction [26]–[29].

7 WRITING ADVICE

This chapter contains a list of common writing mistakes that students tend to make when writing theses. Students should read the list carefully, in addition to the examples provided by GUP in their thesis guidelines, and guard against making these mistakes in their theses.

8 FUTURE WORK

First of all, we plan to finish this template example and, once the style/template has been approved by GUP, disseminate it to others in the CoE. We plan to invite contributions from other CMPE and CoE faculty in improving this document, with the goal to include the most useful information to help students succeed in writing their thesis.

9 CONCLUSIONS

This macro example thesis was written as a guide to CMPE and CoE students for writing quality theses in LaTeX. The provided template, along with this guide, should make the task of complying with GUP formatting requirements much easier for students. The CMPE LaTeX thesis template and this document are a work in progress. The reader is invited to participate in improving both via Git pull requests. The project can be found at https://github.com/davidanastasiu/thesis_template-SJSU_CMPE.

Literature Cited

[1] S. Wang and M. Liu, "Point cloud compression with range image-based entropy model for autonomous driving," in *European Conference on Computer Vision Workshops*, 2022.

[2] T. Fan, Y. Hu, R. Gong, and Y. Wang, "U-motion: Learned point cloud video compression with u-structured temporal context generation," *arXiv preprint arXiv:2411.14501*, 2024.

[3] M. Wang, R. Huang, W. Xie, Z. Ma, and S. Ma, "Compression Approaches for LiDAR Point Clouds and Beyond: A Survey," *ACM Transactions on Multimedia Computing, Communications, and Applications*, vol. 21, pp. 1–31, July 2025.

[4] Y. Xiong, W.-C. Ma, J. Wang, and R. Urtasun, "Learning Compact Representations for LiDAR Completion and Generation," in *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, (Vancouver, BC, Canada), pp. 1074–1083, IEEE, June 2023.

[5] L. Xie, W. Gao, H. Zheng, and G. Li, "ROI-guided point cloud geometry compression towards human and machine vision," in *Proceedings of the 32nd ACM International Conference on Multimedia*, MM '24, 2024.

[6] R. Abbasi, A. K. Bashir, H. J. Alyamani, F. Amin, J. Doh, and J. Chen, "LiDAR point cloud compression, processing and learning for autonomous driving," *IEEE Transactions on Intelligent Transportation Systems*, vol. 24, no. 1, pp. 962–972, 2023.

[7] M. Wang, R. Huang, W. Xie, Z. Ma, and S. Ma, "Compression approaches for LiDAR point clouds and beyond: A survey," *ACM Transactions on Multimedia Computing, Communications, and Applications*, vol. 21, Jul 2025.

[8] N. A. B. Martins, L. A. da Silva Cruz, and F. Lopes, "Impact of LiDAR point cloud compression on 3d object detection evaluated on the KITTI dataset," *EURASIP Journal on Image and Video Processing*, vol. 2024, no. 15, 2024.

[9] M. Isenburg, "LASzip: Lossless compression of LiDAR data," in *Proceedings of the ASPRS Annual Conference*, (Baltimore, MD, USA), 2013.

[10] L. Liu, Z. Hu, and Z. Chen, "Towards point cloud compression for machine perception: A simple and strong baseline by learning the octree depth level predictor," *arXiv preprint arXiv:2406.00791*, 2024.

[11] C. Sun, H. Yuan, S. Jiang, D. Ai, W. Zhang, and R. Hamzaoui, "Lpcm: Learning-based predictive coding for LiDAR point cloud compression," *arXiv preprint arXiv:2505.20059*, 2025.

[12] H. Zheng, W. Gao, Z. Yu, T. Zhao, and G. Li, "ViewPCGC: View-guided learned point cloud geometry compression," in *Proceedings of the 32nd ACM International Conference on Multimedia*, MM '24, 2024.

[13] A. Akhtar, Z. Li, and G. V. der Auwera, "Inter-frame compression for dynamic point cloud geometry coding," *IEEE Transactions on Image Processing*, vol. 33, pp. 584–593, 2024.

[14] J. Li, H. Dai, H. Han, and Y. Ding, "MSeg3D: Multi-modal 3d semantic segmentation for autonomous driving," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023.

[15] X. Sun, M. Wang, J. Du, Y. Sun, S. S. Cheng, and W. Xie, "A task-driven scene-aware LiDAR point cloud coding framework for autonomous vehicles," *IEEE Transactions on Industrial Informatics*, vol. 19, no. 8, pp. 8731–8740, 2023.

[16] M.-F. Chang, W. Dong, J. Mangelson, M. Kaess, and S. Lucey, "Map compressibility assessment for LiDAR registration," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, IROS 2021, 2021.

[17] L. Contreras and W. Mayol-Cuevas, "O-POCO: Online point cloud compression mapping for visual odometry and SLAM," in *Proceedings of the IEEE International Conference on Robotics and Automation*, ICRA 2017, 2017.

[18] C. Tu, E. Takeuchi, A. Carballo, and K. Takeda, "Point cloud compression for 3d LiDAR sensor using recurrent neural network with residual blocks," in *Proceedings of the IEEE International Conference on Robotics and Automation*, ICRA 2019, pp. 3274–3279, 2019.

[19] Y. Feng, S. Liu, and Y. Zhu, "Real-time spatio-temporal LiDAR point cloud compression," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, IROS 2020, 2020.

[20] Y. Cao, Y. Wang, and H. Chen, "Real-time LiDAR point cloud compression and transmission for resource-constrained robots," *arXiv preprint arXiv:2502.06123*, 2025.

[21] Z. Xu, Z. Li, Q. Guan, D. Zhang, Q. Li, J. Nan, C. Liu, W. Bian, and J. Ye, "Large-scale order dispatch in on-demand ride-hailing platforms: A learning and planning approach," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, KDD '18, 2018.

[22] Y. Kim, V. Jayawardana, and S. Samaranayake, "Learning-augmented vehicle dispatching with slack times for high-capacity ride-pooling." SSRN preprint 4801437, Apr 2024. Available at https://ssrn.com/abstract=4801437.

[23] D. C. Anastasiu and G. Karypis, "Efficient identification of tanimoto nearest neighbors," in *2016 IEEE International Conference on Data Science and Advanced Analytics (DSAA)*, DSAA '16, pp. 156–165, 2016.

[24] D. C. Anastasiu, "Cosine approximate nearest neighbors," in *Data Science – Analytics and Applications*, iDSC 2017, (Wiesbaden), pp. 45–50, Springer Fachmedien Wiesbaden, 2017.

[25] M. Kapoor, S. Hua, and D. C. Anastasiu, "Improving student motivation through competitive active learning," in *2018 IEEE Frontiers in Education Conference*, FIE 2018, IEEE, October 2018.

[26] D. C. Anastasiu and G. Karypis, "L2ap: Fast cosine similarity search with prefix l-2 norm bounds," in *The 30th IEEE International Conference on Data Engineering*, ICDE 2014, pp. 784–795, 2014.

[27] D. C. Anastasiu, A. M. Rashid, A. Tagarelli, and G. Karypis, "Understanding computer usage evolution," in *31st IEEE International Conference on Data Engineering*, ICDE 2015, pp. 1549–1560, 2015.

[28] D. C. Anastasiu and G. Karypis, "Pl2ap: Fast parallel cosine similarity search," in *Proceedings of the 5th Workshop on Irregular Applications: Architectures and Algorithms, in conjunction with SC'15*, IA3 2015, (New York, NY, USA), pp. 1–8, ACM, 2015.

[29] D. C. Anastasiu and G. Karypis, "Fast parallel cosine k-nearest neighbor graph construction," in *2016 6th Workshop on Irregular Applications: Architecture and Algorithms (IA3)*, pp. 50–53, Nov 2016.

Appendix A

WHAT SHOULD BE INCLUDED IN APPENDICES

Appendices should contain information that is too lengthy to be included in the thesis chapters but further support the conclusions of the thesis. For example, one could include additional experimental results in the form of tables or additional figures. Each appendix should start with a paragraph introducing the items being presented. Additionally, each table or figure should be preceded by a paragraph that explains the data being presented and the conclusions that can be drawn from them. Please note that data already presented in the main thesis should not be repeated in an appendix. One cannot, for example, include the results of an experiment as a figure in the thesis and again as a table in the appendix. One can, however, include detailed results in tables in the appendix and a summary figure in the thesis portraying only the "best" results.

A.1 Subsections in Appendices

It is possible, and advisable, to use (multiple levels of) subsections in an Appendix. The main Appendix sections may be optionally titled. They will automatically be assigned Appendix A, Appendix B, etc. If a title is present, it will appear on the next line in the appendix and on the same line in the TOC. Sub-sections will be listed in the TOC as A.1, A.2, etc.

Figures, such as Fig. 3, should only be referenced in the appendix. The thesis content can point to an appendix but not specifically reference a table or figure in the appendix.

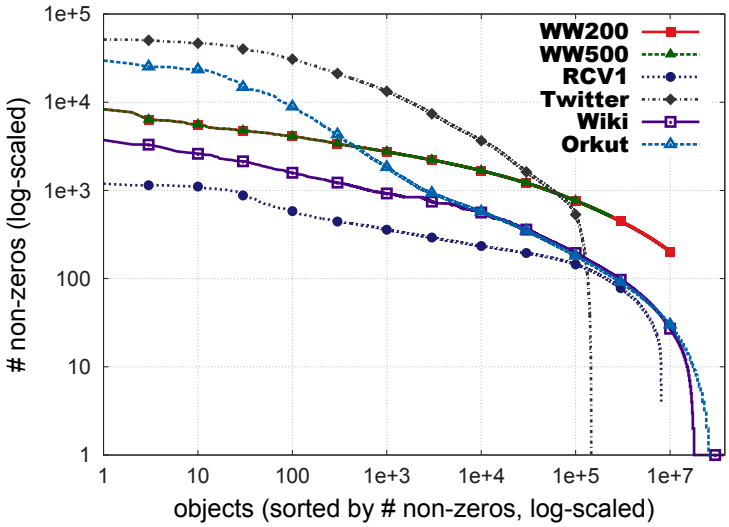

Fig. 3: Distribution of sample non-zeros.

Appendix B

TITLE OF ANOTHER APPENDIX

Different appendices should cover unrelated topics. If the appendices are on the same topic, they should just be sub-sections within the same appendix.

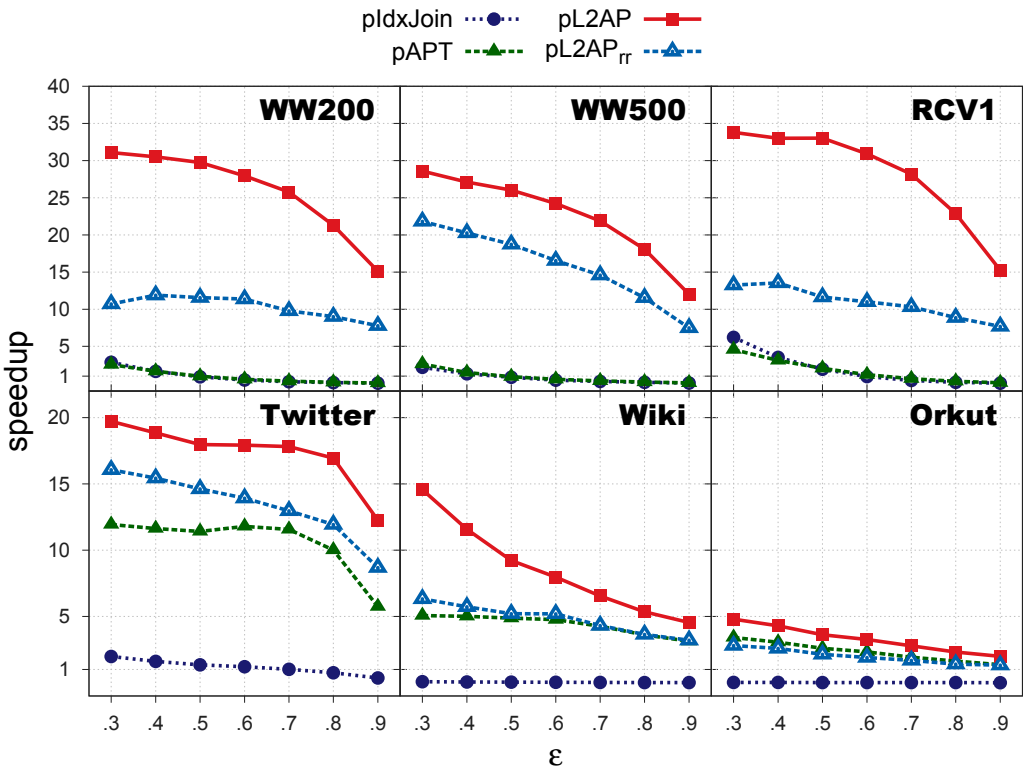

Fig. 4: Speedup comparison of our method against baselines.